Lecture 4 · Part I · Lecture notes

# Training models

Dataset: Titanic, 891 passengers

**How to use these notes**

The primary text for this lecture is Géron, Chapter 4. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

## Contents

**1 Why a closed form is not enough** **2**

**2 Derivation: gradient descent** **3**

2.1 Which way is downhill . . . 3

2.2 The gradient, and the update . . . 3

2.3 How large a step . . . 4

2.4 Why you must scale the features . . . 4

2.5 One instance at a time . . . 4

**3 Fitting by descent** **5**

3.1 Curvature without leaving linearity . . . 5

**4 The worked example: a probability, not a label** **5**

4.1 The data . . . 5

4.2 The label, and four engineered columns . . . 6

4.3 Split first, and measure the anchor . . . 6

**5 Logistic regression** **6**

5.1 The model, and what the linear part predicts . . . 6
5.2 The loss, and why this one . . . 7
5.3 The first numbers . . . 7

**6 Three coefficients that cannot be right** **8**
6.1 Count the columns, then count the rank . . . 8
6.2 The repair . . . 8

**7 Capacity: the degree sweep** **9**

**8 Calibration, and the cut-off nobody chose** **10**

**9 More than two classes** **10**

**10 The notebook** **10**

**11 Exercises** **11**

**Summary** **11**

---

# 1 Why a closed form is not enough

Lecture 3 derived $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal y$, valid exactly when $\mathbf{X}$ has full column rank, and then measured a classifier properly. Both lectures called `.fit()` and moved on. Today we open it.

| Situation | What the normal equation does |
|---|---|
| Many features | inverts an $(n{+}1) \times (n{+}1)$ matrix — cost grows like $n^3$ |
| Many instances | forms $\mathbf{X}^\intercal\mathbf{X}$, so every row must be seen at once |
| Rank-deficient design | the inverse does not exist |
| A cost that is not squared error | there is no closed form at all |

The last row is the one that matters. The classifier we build today minimises a cost whose stationary equations have no solution in elementary functions, so the answer has to be *walked to*.

One dataset runs through the second half: 891 Titanic passengers, and the question how likely each one is to survive. Examinable: the derivation as given here, and logistic and softmax regression — the model, the log loss, coefficients as log-odds, and why a rank-deficient design leaves the fit without a unique answer. Not examinable: the Titanic feature engineering.

## 2 Derivation: gradient descent

You have a cost $J(\boldsymbol{\theta})$ and no formula for its minimiser. The answer must supply three things: a direction to move in, a step size that does not overshoot, and a reason to believe the point you stop at is the minimum rather than merely a place where you stopped.

Throughout: $m$ instances, $n$ features, $\mathbf{X}$ of shape $m \times (n{+}1)$ with a leading column of ones, $\eta > 0$ the **learning rate**, and

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) = \frac{1}{m}\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2.$$

We derive on squared error because it is the one cost whose answer we already have in closed form — which makes it the one cost where the method can be checked against a formula.

### 2.1 Which way is downhill

Move a small distance in a unit direction $\mathbf{u}$. To first order,

$$J(\boldsymbol{\theta} + \epsilon\mathbf{u}) - J(\boldsymbol{\theta}) = \epsilon\,\nabla J(\boldsymbol{\theta}) \cdot \mathbf{u} + o(\epsilon).$$

By Cauchy–Schwarz, $|\nabla J \cdot \mathbf{u}| \le \|\nabla J\|$ for every unit $\mathbf{u}$, with equality only when $\mathbf{u}$ is parallel to $\nabla J$. So

$$\mathbf{u}^* = -\frac{\nabla J(\boldsymbol{\theta})}{\|\nabla J(\boldsymbol{\theta})\|}.$$

That is why the gradient and not some other vector: it is the *unique* direction of steepest decrease, and the argument is three lines long.

### 2.2 The gradient, and the update

**One partial derivative, then stack them**

$$\begin{aligned}\frac{\partial J}{\partial \theta_j} &= \frac{1}{m}\sum_{i=1}^{m} 2\big(\boldsymbol{\theta}^\top\mathbf{x}^{(i)} - y^{(i)}\big)\frac{\partial}{\partial \theta_j}\big(\boldsymbol{\theta}^\top\mathbf{x}^{(i)}\big)\\ &= \frac{2}{m}\sum_{i=1}^{m}\big(\boldsymbol{\theta}^\top\mathbf{x}^{(i)} - y^{(i)}\big)x_j^{(i)},\end{aligned}$$

using $\partial(\boldsymbol{\theta}^\top\mathbf{x})/\partial\theta_j = x_j$, because $\sum_k \theta_k x_k$ contains $\theta_j$ exactly once. Read it as *error* $\times$ *feature*, averaged over the instances. Stacking the partials,

$$\nabla J(\boldsymbol{\theta}) = \frac{2}{m}\,\mathbf{X}^\top\big(\mathbf{X}\boldsymbol{\theta} - y\big),$$

and the update is

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta\,\nabla J\big(\boldsymbol{\theta}^{(k)}\big).$$

One matrix product over the whole training set, at cost $O(mn)$ per evaluation — linear in the rows, where the normal equation was cubic in the columns. No matrix is ever inverted. It is called **batch** gradient descent because every step uses the whole training set.

Note that $\eta$ has units: it converts a gradient into a displacement in parameter space. That is why its right value depends on how the features are scaled.

## 2.3 How large a step

Nothing in the update rule fixes $\eta$. It is a **hyperparameter**, chosen like any other: try a few values and watch the cost.

| Learning rate | What you see |
|---|---|
| too small | the cost falls slowly, and is still falling when you stop |
| about right | the cost drops fast and flattens |
| too large | every step overshoots the minimum: the cost rises, oscillates, and overflows to `nan` |

**Failure condition — divergence is silent until it is loud**

A diverging fit does not raise. It returns weights full of `inf` and a model that predicts one value everywhere. Plot the cost against the step number; it is the only cheap way to see which of the three rows you are in.

## 2.4 Why you must scale the features

Put one feature in thousands and another in units. The cost changes far faster along the first parameter than along the second, and the bowl becomes a long, thin valley. One $\eta$ serves every direction, so it must be small enough not to overshoot along the steep one — and that same $\eta$ is far too small along the shallow one, where progress crawls. The path zig-zags across the valley instead of running down it.

Standardising the columns is not cosmetic. It makes the bowl round, so one learning rate suits every direction and far fewer steps are needed.

## 2.5 One instance at a time

Each batch step costs $O(mn)$, unaffordable for large $m$. Draw an index $i$ uniformly at random and use only that row:

$$\mathbf{g}^{(i)}(\boldsymbol{\theta}) = 2\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\big)\mathbf{x}^{(i)}, \qquad \mathbb{E}_i\big[\mathbf{g}^{(i)}\big] = \nabla J(\boldsymbol{\theta}).$$

The one-instance gradient is an *unbiased* estimate of the batch gradient, and that one line is what makes stochastic and mini-batch descent legitimate rather than merely cheap.

**What the derivation buys you**

That the gradient is error times feature, so a linear model fits without inverting anything; that a rising cost curve is a step-size fault rather than a bug; that scaling is part of the algorithm, not tidiness; and that mini-batches are principled.
And one more: the derivation never used the fact that the cost was squared error until the partial derivative. Swap the cost, recompute the gradient, and the algorithm is unchanged — which is how the classifier later in this lecture is fitted.

# 3 Fitting by descent

| | Normal equation / SVD | Gradient descent |
|---|---|---|
| Cost in $m$ | linear | linear, per step |
| Cost in $n$ | $n^{2.4}$ to $n^3$ | linear |
| Out-of-core | no — needs all rows | yes, with mini-batches |
| Scaling required | no | yes |
| Hyperparameters | none | $\eta$, batch size, schedule, stopping rule |
| Other costs | squared error only | any differentiable cost |

Few features and data that fits in memory: use the closed form, it has no knobs to get wrong. Many features, streaming data, or any cost that is not squared error: descent.

## 3.1 Curvature without leaving linearity

A straight line cannot fit a curve, but $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$ is linear in $\boldsymbol{\theta}$ even though it is not linear in $x$. Everything derived above applies unchanged — the gradient, the update, the case for scaling — because all of them were statements about $\boldsymbol{\theta}$.

`PolynomialFeatures(degree=d)` also generates every cross product, which is how a linear model represents an interaction such as "third class and travelling alone". The column count is $\binom{n+d}{d}$, so "a bit more capacity" is never a small change.

# 4 The worked example: a probability, not a label

A marine safety planning unit wants an evacuation-assistance model: for each passenger profile, how likely that person is to get off unaided — and they need to explain the assignment afterwards.

| What they asked for | What that forces |
|---|---|
| "how likely" | a *probability*, not a label |
| the number is used to allocate | it must be *calibrated* |
| "explain the assignment" | weights a human can read |

**Why requirement 1 is not a formatting preference**

"This passenger survives" is not checkable on its own. "This passenger survives with probability 0.31" is: take everyone the model puts near 0.31 and count.

## 4.1 The data

891 passengers, 12 columns: the whole labelled corpus. The archive also holds a `test.csv` with no `Survived` column — a competition submission file, useless for measuring anything — so we hold out our own test set.

### 4.2 The label, and four engineered columns

| Outcome | Passengers | Share |
|---|---|---|
| Did not survive | 549 | 61.6% |
| Survived | 342 | 38.4% |

Mildly imbalanced — nothing like the rare-event problem of Lecture 3. Accuracy will not be absurd here. It will be something more dangerous than absurd: plausible.

Four engineered columns are added, each with a reason: `Title`, extracted from the name as Mr, Mrs, Miss, Master or Rare (a boy is not a man), `FamilySize` (a group evacuates together, and slowly), `IsAlone` (nobody is waiting for you), `Deck` (distance to a boat, and a proxy for class; `U` where no cabin was recorded). Note that `FamilySize = SibSp + Parch + 1` is an exact linear function of two columns we already have. Remember it — we come back to it with a rank computation, not with an opinion.

### 4.3 Split first, and measure the anchor

Stratified on the *label* this time, not on a predictor: with 179 test rows an unstratified draw can shift the base rate by several points, and every downstream number moves with it. That gives 712 training and 179 test passengers, at rates 0.3834 and 0.3855. 712 is a small dataset, and by the end of the lecture the size is doing visible damage.

With $p = 0.3834$, a model that has learned nothing except the base rate scores $1 - p = 0.617$ accuracy and a log loss of

$$-\big(p \log p + (1-p)\log(1-p)\big) = 0.666$$

— the entropy of the label, in nats. Every number below has to be compared against it.

## 5 Logistic regression

### 5.1 The model, and what the linear part predicts

$$\hat{p} = \sigma\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}\big), \qquad \sigma(t) = \frac{1}{1+e^{-t}}.$$

$\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$ is exactly the quantity least squares fitted; everything new is in $\sigma$ and in what we ask $\boldsymbol{\theta}$ to minimise. $\sigma$ is strictly increasing and maps $\mathbb{R}$ onto $(0, 1)$, so it never returns exactly 0 or 1 — which matters when the loss takes a logarithm of it. It is monotone, so the ranking is the linear model's ranking; and it saturates, so beyond about $|t| = 5$ more evidence changes almost nothing.

Inverting it:

$$t = \log\frac{\hat{p}}{1-\hat{p}} = \boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}.$$

**The sentence to get right**

The linear model does not predict the probability. It predicts the *log-odds*, and the coefficients act on the *odds*: add 1 to $x_j$ and the odds are multiplied by $e^{\theta_j}$, wherever you started. The probability has no such rule.

## 5.2 The loss, and why this one

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\Big[y^{(i)}\log\hat{p}^{(i)} + \big(1-y^{(i)}\big)\log\big(1-\hat{p}^{(i)}\big)\Big].$$

Only the probability given to the true class enters each term, and it is unbounded above: claim 0.001 for a passenger who survived and that term is $-\log 0.001 \approx 6.9$.

It is convex in $\boldsymbol{\theta}$, by the argument Lecture 2 used for the MSE: its Hessian is $\mathbf{H} = \frac{1}{m}\sum_i \hat{p}^{(i)}\big(1-\hat{p}^{(i)}\big)\mathbf{x}^{(i)}\mathbf{x}^{(i)\mathsf{T}}$, and for any $\mathbf{v}$, $\mathbf{v}^\mathsf{T}\mathbf{H}\mathbf{v} = \frac{1}{m}\sum_i \hat{p}^{(i)}\big(1-\hat{p}^{(i)}\big)\big(\mathbf{v}^\mathsf{T}\mathbf{x}^{(i)}\big)^2 \geq 0$. So every point where the gradient vanishes is a global minimum — though, as we will see, not necessarily the only one.

A scoring rule is **proper** if the expected score is optimised by reporting your true belief. Log loss is proper; so is the Brier score $\frac{1}{m}\sum(\hat{p}-y)^2$, which is bounded and easier to read, and we report it too.

**Failure condition — accuracy is not proper**

Under accuracy a model reporting 0.51 scores exactly the same as one reporting 0.99. So accuracy gives a model no reason at all to tell the truth about its uncertainty, and optimising it destroys calibration.

## 5.3 The first numbers

The numeric columns are imputed and standardised, the categorical ones one-hot encoded, all inside the pipeline, so nothing is fitted on a held-out fold; standardising is the scaling step of the derivation, applied, because the solver is doing descent. The folds are a `StratifiedKFold`, not a `KFold`: with 71 passengers per fold, an unstratified split can hand a fold a base rate several points from the truth. The model is fitted with `C=1e6`, which turns Scikit-learn's default penalty (`C=1.0`) *off*, so today's coefficients are the data's rather than a compromise with a penalty. Regularisation is the next lecture.

| Model | Log loss | Accuracy |
|---|---|---|
| Report the base rate to everyone | 0.666 | 61.7% |
| Logistic regression, 10-fold cross-validated | 0.496 | 81.5% |

About a quarter below the anchor on log loss and twenty points of accuracy above the majority class. And the per-fold spread of the log loss is 0.22 — larger than any difference this lecture compares. Read every later gap against it.

# 6 Three coefficients that cannot be right

`Title_Miss` at −2.81, when 70.3% of them survived against 38.4% overall. `Title_Mrs` at −2.02, the highest-surviving group in the data. And `Deck_T` is the largest weight in the model, when there is exactly one passenger on deck T.

These are not weak effects or noisy estimates. They have the wrong *sign*, and the model's predictions are nonetheless good. Before reaching for a story about confounding, ask the cheaper question: what is the shape of the design matrix?

## 6.1 Count the columns, then count the rank

```
print("columns:", Z_b.shape[1])            # 29
print("rank:   ", np.linalg.matrix_rank(Z_b))  # 23
print("cond:   ", np.linalg.cond(Z_b))         # about 1e16
```

Six columns are linear combinations of the others. Five come from the encoder — each one-hot block sums to 1 in every row, and the intercept column is also 1 in every row, so five categorical columns give five exact dependencies. The sixth is ours: `FamilySize = SibSp + Parch + 1`, exactly, for every passenger, with maximum residual 0.0. Not "highly correlated" — an identity. Standardising does not help, because an affine map of a linear dependence is still a linear dependence.

**Failure condition — what a rank-deficient design does to the fit**

If $\mathbf{X}\mathbf{v} = \mathbf{0}$ for some $\mathbf{v} \neq \mathbf{0}$, then $\mathbf{X}(\boldsymbol{\theta} + c\mathbf{v}) = \mathbf{X}\boldsymbol{\theta}$ for every $c$: along $\mathbf{v}$ the loss is flat. There is not one best $\boldsymbol{\theta}$ but a whole affine subspace of them. The solver returns one point of it and says nothing. Predictions are unaffected; *every claim about a coefficient is void*.

Proved on our own matrix: add 2.5 to `Sex_female` and to `Sex_male`, subtract 2.5 from the intercept, and the largest change in any of the 712 logits is of order $10^{-15}$, rounding noise. The two coefficient vectors differ by 2.5 in three places, and both fits make the same prediction for every passenger.

**The test, stated generally**

If a coefficient can be moved without changing any prediction, that coefficient is not a property of the data.

## 6.2 The repair

Drop one level per categorical block and remove `FamilySize`. That is `drop="first"`; the similar-looking `drop="if_binary"` drops a level only from two-level features such as `Sex`, and leaves every block of three or more levels whole, still summing to the intercept column.

| Design matrix | Columns | Rank | Condition number |
|---|---|---|---|
| As first written | 29 | 23 | about $10^{16}$ |
| One level dropped, `FamilySize` removed | 23 | 23 | 83.6 |

Fourteen orders of magnitude. `IsAlone` stays: it is an indicator, not a linear function of the counts.

Each dropped level becomes the **reference**, folded into the intercept, and every remaining coefficient is read *relative to it*. Here the references are first class, female, Cherbourg, *Master* and deck A. A coefficient quoted with no stated reference level cannot be read by anybody, including you in a month.

Use `min_frequency` rather than `handle_unknown="ignore"` here: with `drop="first"` an ignored unknown level is encoded all-zeros, which is the encoding of the reference level, so our one deck-T passenger would be scored as though they were on deck A whenever a fold holds them out.

Now a coefficient can be read aloud. `Pclass_3` $= -1.169$: holding every other recorded feature fixed, travelling third class multiplies the odds of survival by $e^{-1.169} = 0.31$, relative to first class. Every clause matters — "holding every other feature fixed" is doing real work: drop it and the sentence is false; the multiplier is on odds rather than probability (odds of 1 multiplied by 0.31 give $p = 0.24$, but starting from $p = 0.9$ the same multiplier gives 0.74); and it is always relative to the reference level. And unique is not the same as stable: `Sex` and `Title` still carry much of the same information, so how the effect splits between them moves with the sample. We only fixed uniqueness.

## 7 Capacity: the degree sweep

Add polynomial features, degrees 1 to 6, to the four numeric columns only: squaring a one-hot indicator returns the indicator, and the repair above has just removed exact copies. The model is still linear in $\boldsymbol{\theta}$.

| Degree | Columns | Training | Held-out | Held-out accuracy |
|---|---|---|---|---|
| 1 | 22 | 0.395 | 0.468 | 81.5% |
| 2 | 32 | 0.381 | 0.467 | 82.2% |
| 3 | 52 | 0.355 | 0.538 | 79.6% |
| 4 | 87 | 0.310 | 1.390 | 77.4% |
| 5 | 143 | 0.286 | 1.922 | 76.0% |
| 6 | 227 | 0.284 | 1.836 | 76.7% |

At degree 6 the model has 227 weights to determine from 712 passengers, about three rows per weight. The training curve never stops improving; the held-out curve is flat across degrees 1 and 2, turns after degree 2 and then falls off a cliff. Degrees 1 and 2 differ by 0.001 in held-out log loss, and the smaller of their fold-to-fold spreads, 0.12, is 117 times that. Those two are a *tie*, and the curve turns *after* degree 2. Crowning either one — degree 2 has the lower mean — would be reporting noise.

From degree 2 to degree 5 held-out accuracy falls by 6.2 points while held-out log loss is multiplied by more than four. Accuracy only notices when a prediction crosses the cut-off; log loss notices a confident prediction becoming a confidently *wrong* one. The failure is not that the model gets more passengers wrong — it is that it becomes certain about the ones it gets wrong. The requirement was for probabilities, and the metric that matches the requirement is the metric that saw the damage.

## 8 Calibration, and the cut-off nobody chose

**Definition 8.1** (Calibration)**.** $P(y = 1 \mid \hat{p}(\mathbf{x}) = q) = q$ *for every* $q \in (0, 1)$.

Take every passenger the model scored near $q$; the fraction who actually survived should be about $q$. This is a property of a *set* of predictions, not of any one prediction — which is exactly why it is so easy to ship a model that fails it.

Check it out of fold, with `cross_val_predict` rather than `predict`, so every probability came from a fit that had never seen that passenger. A reliability diagram drawn on training predictions always looks excellent.

The model is close to the diagonal in the bins that hold most of the passengers, and that is not a happy accident: log loss is a proper scoring rule and we minimised it directly, so calibration is what the objective was asking for.

**Failure condition — a bare label hides a decision nobody made**

`.predict()` is `.predict_proba() >= 0.5`. That cut-off is cost-optimal only when the two mistakes cost the same. Nothing in the code mentions 0.5, nothing errors, and the accuracy printed is true — it just answers a question the stakeholder did not ask.

Lecture 3 chose an operating point against a stated constraint and defended it. The same discipline applies here, with one addition: the model produces the probability, and the cut-off is a separate decision taken on top of it.

## 9 More than two classes

Softmax regression gives one parameter vector per class:

$$s_k(\mathbf{x}) = \boldsymbol{\theta}_k^{\mathsf{T}}\mathbf{x}, \qquad \hat{p}_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} \exp(s_j(\mathbf{x}))},$$

so the output is a distribution over the $K$ classes by construction. It is multi*class*, not multi*label*: four objects in one photograph need $K$ separate binary classifiers, not this.

Its cost is cross-entropy,

$$J(\boldsymbol{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)},$$

whose inner sum keeps exactly one term. At $K = 2$ this is the log loss, term for term: logistic regression is the two-class case of softmax, not a different model.

## 10 The notebook

It runs on a free CPU.

**What to change**

Gradient descent is implemented from the derivation in a dozen lines, checked against the normal equation to eight decimals, and run at three learning rates, one of which diverges; then the Titanic manifest as in the lecture — the anchor, the rank computation and its repair, the coefficients, the degree sweep, the calibration bins, and softmax on three classes. Lower `ETA_DIVERGE` until the cost stops blowing up, and find roughly the largest learning rate that still converges — which is how a learning rate is chosen in practice.

## 11 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 5's deck and in the exercise book.

1. Gradient descent on the same data, with the features unscaled, takes many more steps to converge than on scaled features. Explain why, in terms of the shape of the cost surface. [5]
2. On the Titanic manifest, your logistic regression on one-hot encoded columns produces coefficients that look wrong, and the design matrix has rank six less than its column count. State the cause and the standard repair. [5]
3. A model reaches 0.80 accuracy and is *badly calibrated*. Explain what that means, and name a decision it would make wrongly that accuracy cannot see. [4]
4. In a logistic regression with first class as the reference level, the coefficient on `Pclass_3` is $-1.169$. State what it multiplies, by how much, and why no single change in the survival probability follows from it. [4]
5. State the update rule for batch gradient descent, and say what changes in it for stochastic gradient descent — and what does not. [4]

## Summary

Descent moves along $-\nabla J$, which Cauchy–Schwarz identifies as the steepest direction. On the MSE that gradient is $\frac{2}{m}\mathbf{X}^{\mathsf{T}}(\mathbf{X}\boldsymbol{\theta} - y)$, so a linear model is fitted without ever forming an inverse, and it reaches the normal equation's answer. The learning rate is a hyperparameter, set by watching the cost curve, and unscaled features turn the bowl into a thin valley that slows descent down, which is why scaling is part of the algorithm. One row gives an unbiased gradient, which is what makes mini-batches legitimate.

Logistic regression predicts the log-odds linearly and minimises a proper scoring rule, by the same descent. Its fit here was *not unique*: six of 29 columns were linear combinations of the others, shown by shifting two coefficients by 2.5 and moving no prediction. Repairing the encoding took the condition number from about $10^{16}$ to 83.6, and made the coefficients readable — relative to a stated reference level. Adding polynomial capacity made the training curve fall forever while the held-out log loss turned after degree 2.

And a probability is checkable where a label is not. The out-of-fold probabilities are close to calibrated, and `.predict()` applies a cut-off of 0.5 that nobody chose.